Smarter Fuzzing: Using Constants and Static Analysis to Find Errors in JPAMB

ARI GUNNAR KRISTJÓNSSON, BRYNJA EYFJÖRÐ JÓNSDÓTTIR, and EGILL MAGNÚSSON, DTU Compute - Group 23, Denmark

1 INTRODUCTION

Context. In this course we have built analyses that try to answer questions about Java methods, like can this method divide by zero, can an assertion fail, or can it loop forever. The JPAMB benchmark[1] has a set of small Java methods with these questions, and gives a score based on how good the analysis predicts what can happen.

So far we have tried two kinds of analysis. A static analysis, like our abstract interpreter, looks at all possible executions at once. If it never reach an outcome, the outcome can not happen, so we can say "no" with confidence. But if it does reach it, we don't know if it is a real error or only because the abstraction is not precise enough. A dynamic analysis does the opposite and runs the method on real inputs. When it finds an error, the error is real and we have the input that caused it. But when it finds nothing, it does not prove anything.

Gap. The problem is that errors are often hidden behind specific values, like in listing 1. In the first method, the division is only by zero when $n = 10054203$. An interval analysis can see that the divisor could be zero, but it can only say "maybe". A random fuzzer picks from all 2^{32} integers, so the chance to hit the right value is 2^{-32} for each try. Our dynamic analysis from earlier only tries small values close to zero, so it never finds it. So none of them can say "yes".

The second method shows another problem, this time with the benchmark itself. JPAMB gets the correct answers from the inputs that are listed for each case, and for `divisionLoop` only $n = 0$ and $n = 1$ are listed. Both of them terminate normally. So the assertion error for $n = 1024$ is real, but the benchmark says it can not happen. This means an analysis can get a high score just by fitting the listed cases, for example a syntactic analysis that looks for patterns in the code, without really knowing anything about the program. Because of this we don't think the score alone says very much.

Innovation. Our idea is to not guess the inputs, but to find them from the program itself. Each analysis gives information to the next one:

(1) A syntactic analysis finds the constants in the bytecode, like `10054203` and `32`.
(2) A static analysis with intervals and widening runs until it reaches a fixpoint. Outcomes it never reach are answered with "no", and we don't need to search for them.
(3) For the outcomes that are left, an input generator finds the instruction that can cause it (a division, an array access or the check before an assertion). Then it goes backwards through the arithmetic between the parameter and that instruction, and inverts it (addition, subtraction, multiplication and division) to solve for the input. For listing 1 this gives $n = 10054203$ for the first method and $n = 32 \cdot 2^5 = 1024$ for the second. If it can not invert the arithmetic, it falls back to the constants, their neighbours $c - 1, c, c + 1$, and values like 0, ± 1 and the smallest and largest int.

[1] https://github.com/kalhauge/jpamb

Authors' address: Ari Gunnar Kristjónsson, s253023@dtu.dk; Brynja Eyfjörð Jónsdóttir, s221901@dtu.dk; Egill Magnússon, s253025@dtu.dk, DTU Compute - Group 23, Kgs. Lyngby, Denmark.

Listing 1. Two JPAMB methods where the error only happens for specific values.

```
public static int divideByNMinus10054203(int n) {
  return 1 / (n - 10054203); // divides by zero only for n = 10054203
}

public static void divisionLoop(int n) {
  for (int i = 0; i < 5; i++) {
    n /= 2;
  }
  assert n != 32; // fails only for 1024 <= n <= 1055
}
```

(4) A concrete interpreter runs the inputs. If an input causes the outcome, we answer "yes" and we have the input as proof.

The whole thing is deterministic, and every confident answer has a reason behind it: a proof for "no" and an input for "yes".

It is clear that solving for `10054203` is better than guessing it, so the interesting part is not if the approach works on cases like listing 1, but how far it gets and where it stops working. Inverting the arithmetic gets harder with loops that run an unknown number of times, with conditions on more than one parameter, with nonlinear arithmetic like $x \cdot y$, and with arrays. We want to find out where the line is.

We want to answer these research questions:

RQ1 Which kinds of conditions can the generator solve, and where does it have to fall back to guessing or give up?

RQ2 How much does the static analysis help, in fewer runs and fewer "maybe" answers?

RQ3 How does the pipeline compare with random fuzzing, small value testing and the syntactic analysis on the JPAMB methods, that we did not write ourself?

From the assignments we already have a concrete interpreter, a sign analysis and a small value tester. New in the project are:

- an interval abstract interpreter with widening for JPAMB, and a proof that the widening terminates,
- an input generator that inverts arithmetic to find inputs that cause errors,
- a pipeline where the syntactic and static results guide the dynamic analysis,
- new benchmark methods, sorted by the kind of condition that guards the error,
- a comparison with random fuzzing, small value testing and syntactic analysis.

Evaluation. We use two sets of methods. The first is the JPAMB suite. We did not write it and it is not made with our approach in mind, so it is the most fair test, and we use it to answer RQ2 and RQ3. If other groups add methods to JPAMB before 16 November, we test on them too, since we did not write them either.

The second set is around 20 methods we write ourself, in categories: magic constants, chains of arithmetic, loops that change the input, conditions on two parameters, nonlinear arithmetic, and arrays. For some categories we expect the generator to work and for others we expect it to fail, and we write about the same number of methods in each. We only use this set for RQ1, to show the limits of the approach, not to show that it wins. We will also submit 5 of them to JPAMB before 16 November, as the EXB contribution.

We compare the full pipeline with two kinds of other analyses. The baselines are the other ways to answer the same questions: our best syntactic analysis from the first assignment, which already gets a high JPAMB score just by fitting the cases, random fuzzing, and small value testing (our dynamic analysis). The ablations are the pipeline with one part taken out: the generator with only the constants, and the pipeline without the static analysis. These are not meant to be strong, they show how much each part adds (RQ2).

Because the JPAMB score can get close to the max, the score alone does not show much difference. So we also measure how many runs it takes before the first error is found, how many answers are still "maybe", how many answers are backed by a proof or an input, and the running time. The syntactic analysis can score high, but none of its answers are backed by anything. The random baseline is run 10 times and we report the mean and standard deviation. If we find an error that the benchmark says is impossible, like in `divisionLoop`, we report it separately.

The biggest threat to validity is that both sets are small and made up, and that we write the second set ourself, so it can be biased towards our approach. We report the results for every category, also the ones where it fails.

A PLAN

Table 1 shows the technical contributions and how we split the points. Figure 1 shows the plan. First we write the new benchmark methods, so they are done before the input generator exists. Then we build the interval analysis and the generator at the same time, put them together in the pipeline, and run the evaluation. We want to get new methods into JPAMB before 16 November.

Fig. 1. Project Timeline

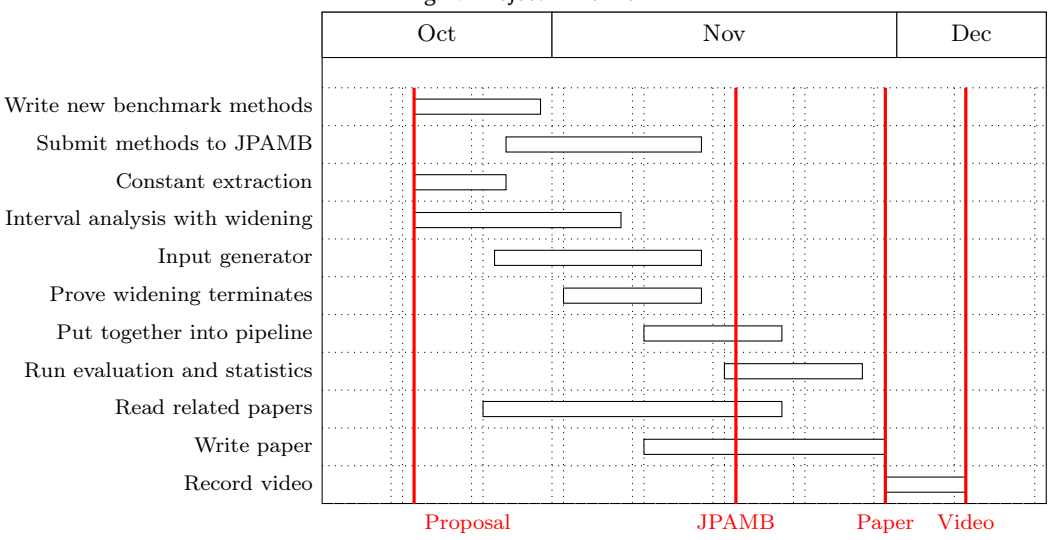


Table 1. Contributions, S_1 is Ari (s253023), S_2 is Brynja (s221901), S_3 is Egill (s253025).

ID	Contribution	P	S_1	S_2	S_3
ISY	Implement Syntactic Analysis (constant extraction)	5	5	0	0
IAB	Implement Novel Abstractions (intervals)	5	0	5	0
IBA	Implement Unbounded Static Analysis	7	0	7	0
NAN	Integrate Analyses (3)	10	4	2	4
EXB	Extend Benchmark Suite	5	1	1	3
EST	Do Statistical Analysis of Results	5	0	0	5
TWI	Prove termination for widening (intervals)	5	0	2	3
PWP	Write Paper	8	3	3	2
PEX	Find Illustrating Examples	4	2	0	2
PDI	Create a Process Diagram	3	3	0	0
PRW	Read and relate to recent papers	3	2	0	1
Total		60	20	20	20
Suggested Max Grade		-	12	12	12